Codex Course for David

Codex Course Part 4 - Review, Automation, and Reference

Review, testing, automations, subagents, hooks, rules, memories, final exam, and quick reference.

Source note: built from the DOV Codex Course - AI Wisebase Learning Pack. Official Codex source refreshed on 2026-06-29.

Module 10 - Review, Testing, and Git

Codex should not just make changes. It should verify them.

Ask for:

- smallest relevant test
- lint or type check when appropriate
- manual verification steps
- review of the diff before acceptance
- file and line references for findings

Practice:

```
Review the uncommitted changes like a serious code reviewer. Findings first, ordered by
severity, with exact file references. If there are no issues, say so and name any test gaps.
```

Mastery check:

- David can ask for a review in a way that produces actionable findings rather than vague commentary.

Sources:

- https://developers.openai.com/codex/app/review
- https://developers.openai.com/codex/integrations/github

Module 11 - Automations

Automations run recurring Codex tasks. They can add findings to the inbox or archive themselves if nothing matters.

Use a thread automation when the task should wake up the same conversation. Use a standalone/project automation when each run should be independent.

Good automation prompt ingredients:

- What to check.
- Where to check.

- Cadence.
- What counts as a finding.
- What to ignore.
- Whether to edit, report only, or ask first.
- When to stop.

Practice:

```
Draft an automation prompt that checks this vault weekly for stale Codex learning notes. It
should report only, not edit.
```

Mastery check:

- David can distinguish "remind me in this thread" from "run a recurring project check."

Sources:

- https://developers.openai.com/codex/app/automations

Module 12 - Subagents, Hooks, Rules, and Memories

Advanced features:

- Subagents: parallel workers for read-heavy exploration, test triage, summaries, or independent reviews.
- Hooks: scripts that run at lifecycle events such as before tool use, after tool use, before compaction, or when a turn stops.
- Rules: command policy that controls what Codex can run outside the sandbox.
- Memories: local recall layer for stable preferences and recurring workflows.

Use these only when the simpler surface is not enough.

Practice:

```
Use parallel subagents to inspect three separate areas of this vault: Codex notes, Obsidian
integration notes, and prompt-library skills. Return one merged summary with file paths.
```

Mastery check:

- David can explain when advanced machinery saves time and when it adds risk or complexity.

Sources:

- https://developers.openai.com/codex/subagents
- https://developers.openai.com/codex/hooks
- https://developers.openai.com/codex/rules
- https://developers.openai.com/codex/memories

Four-Week Course Schedule

Week 1 - Core Use

- Day 1: Module 1
- Day 2: Module 2
- Day 3: Module 3
- Day 4: Module 4

- Day 5: Review flashcards and do one read-only vault inspection

Outcome: David can prompt Codex safely and understand what it is doing.

Week 2 - Durable Setup

- Day 1: Module 5
- Day 2: Module 6
- Day 3: Module 7
- Day 4: Module 8
- Day 5: Create or revise one small skill draft

Outcome: David can decide where repeated instructions and capabilities belong.

Week 3 - Real Tools

- Day 1: Module 9
- Day 2: Module 10
- Day 3: Practice Browser on a local or public page
- Day 4: Practice review mode on a small change
- Day 5: Explain the workflow back to AI Wisebase

Outcome: David can test, inspect, and review real work without guessing.

Week 4 - Automation and Advanced Work

- Day 1: Module 11
- Day 2: Module 12
- Day 3: Draft one reporting-only automation
- Day 4: Draft one subagent task
- Day 5: Final practical exam

Outcome: David can turn repeated work into durable systems.

Final Practical Exam

Ask Codex:

```
I want to improve my Codex learning system in this DOV vault.

Goal: Inspect the current Codex, AI, prompt-library, and integration notes. Create one small
improvement that helps me learn Codex better.
Context: This is an Obsidian vault at C:\ROOT_OBSIDIAN\DOV.
Constraints: Keep edits minimal. Do not reorganize the vault. Do not touch private files. Use
official Codex docs for product claims.
Done when: You create or update one learning note, explain what changed, and give me one next
practice exercise.
```

Passing answer:

- Codex inspects before editing.
- It keeps scope small.
- It uses current official documentation for product claims.
- It creates a useful note.
- It reports what changed and how to practice next.

Final Quiz - Surface Choice

For each need, choose the best surface:

1. "Remember this repo's test command every time."
2. "Run this recurring weekly report."
3. "Connect Codex to a documentation server."
4. "Package a reusable workflow for installation."
5. "Click through a visible Windows app."
6. "Preview a local unauthenticated web app."
7. "Store a repeated task workflow."
8. "Do a parallel read-heavy investigation."

Answer key:

1. AGENTS.md
2. Automation
3. MCP
4. Plugin
5. Computer Use
6. Browser
7. Skill
8. Subagents

Quick Reference

Need	Use
One-time task constraint	Prompt
Unclear task	Plan mode
Long task with success criteria	Goal mode
Repo-specific durable guidance	AGENTS.md
Personal or project settings	config.toml
Reusable workflow	Skill
Installable bundle of skills/apps/MCP	Plugin
External live tools or context	MCP or connector
Local unauthenticated web preview	Browser
Logged-in Chrome state	Chrome
Desktop GUI operation	Computer Use
Recurring check	Automation
Command permission policy	Rules
Lifecycle script	Hook
Stable personal recall	Memories
Parallel read-heavy investigation	Subagents

Maintenance Rule

Refresh this course when Codex features, plugin availability, or OpenAI docs change. Product claims should come from current official docs, not from memory alone.